**TECHNICAL UNIVERSITY OF MOLDOVA**
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

# LABORATORY REPORT

## Lab 9: Watching System Calls with strace

**Operating Systems**

**Student:** Gafenco Victor

**Group:** FAF-241

**Instructor:** Assistant lecturer, Patricia Reițman

**Chișinău - 2026**

## CONTENTS

**Abstract** . . . . . **3**

**Introduction** . . . . . **4**

**1 Tracing a real command** . . . . . **5**

1.1 Output and directory reads . . . . . 5

**2 A program that calls the kernel** . . . . . **6**

2.1 Writing and reading a file . . . . . 6

2.2 Interpreting the trace . . . . . 6

**Conclusions** . . . . . **7**

## ABSTRACT

Linux system-call activity was observed with `strace`, first for the `ls -l` command and then for a C program that opens, writes and reads a file. The experiments identify the calls used for terminal output, explain file descriptor allocation, and interpret the selected system-call count.


**Keywords:** Linux, system calls, strace, openat, read, write, file descriptors

## INTRODUCTION

This laboratory observes the system-call boundary on Linux. The first experiment traces `ls -l` while filtering `openat`, `read`, `write` and `close`. The second uses a small C program to create `note.txt`, write “hello, kernel” to it, read it back and display the result. The program was compiled with GCC using `-Wall -Wextra`.

For the filtered `ls -l` pipeline, `wc -l` counted 187 displayed calls. This is an observed count for the selected call types and this machine, not a count of every system call made by the process. The accompanying screenshots and full `strace` excerpt are in the lab directory.

# 1 TRACING A REAL COMMAND

## 1.1 Output and directory reads

The trace shows `ls` printing the listing with `write(1,…)`. Descriptor 1 is standard output, which is the terminal in the direct run. In the counting pipeline, shell redirection sends standard output to `/dev/null`; the trace itself is redirected to standard error and piped to `wc -l`.

The trace also includes `openat`, `read` and `close` calls as `ls` accesses directory and locale data. Several optional locale-catalog lookups return `ENOENT`. These are unsuccessful probes for files that are absent in the environment; they do not show recursive directory traversal. `ls -l` without a recursive option lists the requested directory only.

```
close(4)                                = 0
openat(8, "lib", O_RDONLY|O_NOFOLLOW|O_CLOEXEC|O_PATH) = 4
close(8)                                = 0
openat(4, "userdb", O_RDONLY|O_NOFOLLOW|O_CLOEXEC|O_PATH) = -1 ENOENT (No such file or directory)
close(4)                                = 0
close(5)                                = 0
close(6)                                = 0
close(7)                                = 0
close(3)                                = 0
openat(AT_FDCWD, "/usr/share/locale/en_US.UTF-8/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en_US.utf8/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en_US/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en.UTF-8/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en.utf8/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/en/LC_MESSAGES/coreutils.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
write(1, "total 0\n", 8total 0
)               = 8
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = 3
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\20\0\0\0\0"..., 4096) = 2424
read(3, "TZif2\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\20\0\0\0\20\0\0\0\0"..., 4096) = 1509
close(3)                                = 0
write(1, "drwxr-xr-x. 1 victor victor  312"..., 55drwxr-xr-x. 1 victor victor  312 Nov  8 12:26  Desktop
) = 55
write(1, "drwxr-xr-x. 1 victor victor   20"..., 57drwxr-xr-x. 1 victor victor   20 Sep  8 11:16  Documents
) = 57
write(1, "drwxr-xr-x. 1 victor victor 2598"..., 57drwxr-xr-x. 1 victor victor 2598 Oct 28 15:42  Downloads
) = 57
write(1, "drwxr-xr-x. 1 victor victor    0"..., 55drwxr-xr-x. 1 victor victor    0 Sep  9 17:59  install
) = 55
write(1, "drwxr-xr-x. 1 victor victor   34"..., 62drwxr-xr-x. 1 victor victor   34 Sep 14 15:17  kwallet_backup
) = 62
write(1, "drwxr-xr-x. 1 victor victor    0"..., 53drwxr-xr-x. 1 victor victor    0 Sep  1 16:29  Music
) = 53
write(1, "drwxr-xr-x. 1 victor victor  272"..., 57drwxr-xr-x. 1 victor victor  272 Nov  8  2026  Nextcloud
) = 57
write(1, "drwxr-xr-x. 1 victor victor   86"..., 62drwxr-xr-x. 1 victor victor   86 Sep 17 12:34  pdf-tools-venv
) = 62
write(1, "drwxr-xr-x. 1 victor victor   22"..., 56drwxr-xr-x. 1 victor victor   22 Sep  4 13:51  Pictures
) = 56
write(1, "drwxr-xr-x. 1 victor victor  174"..., 56drwxr-xr-x. 1 victor victor  174 Nov  4 18:59  Projects
) = 56
write(1, "drwxr-xr-x. 1 victor victor    0"..., 54drwxr-xr-x. 1 victor victor    0 Sep  1 16:29  Public
) = 54
write(1, "drwxr-xr-x. 1 victor victor   52"..., 61drwxr-xr-x. 1 victor victor   52 Oct  6 10:06  RiderProjects
) = 61
write(1, "drwxr-xr-x. 1 victor victor   48"..., 54drwxr-xr-x. 1 victor victor   48 Oct 12 10:10  shtest
) = 54
write(1, "drwxr-xr-x. 1 victor victor    0"..., 57drwxr-xr-x. 1 victor victor    0 Sep  1 16:29  Templates
) = 57
write(1, "drwxr-xr-x. 1 victor victor    0"..., 54drwxr-xr-x. 1 victor victor    0 Sep  1 16:29  Videos
) = 54
write(1, "drwxr-xr-x. 1 victor victor   32"..., 63drwxr-xr-x. 1 victor victor   32 Sep 28 17:12 'VirtualBox VMs'
) = 63
write(1, "drwxr-xr-x. 1 victor victor  264"..., 57drwxr-xr-x. 1 victor victor  264 Oct 12 10:18  xv6-riscv
) = 57
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
victor@localhost:~> strace -f -e trace=openat,read,write,close ls -l 2>&1 > /dev/null | wc -l
187
victor@localhost:~>
```

**Figure 1.1 – Selected calls from `strace` and the 187-line filtered count**

# 2 A PROGRAM THAT CALLS THE KERNEL

## 2.1 Writing and reading a file

The submitted `syscalls.c` calls `open` with `O_CREAT | O_WRONLY | O_TRUNC`, writes the 14-byte message, closes the descriptor, then opens the file read-only and reads it back. The program prints:

```
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
```

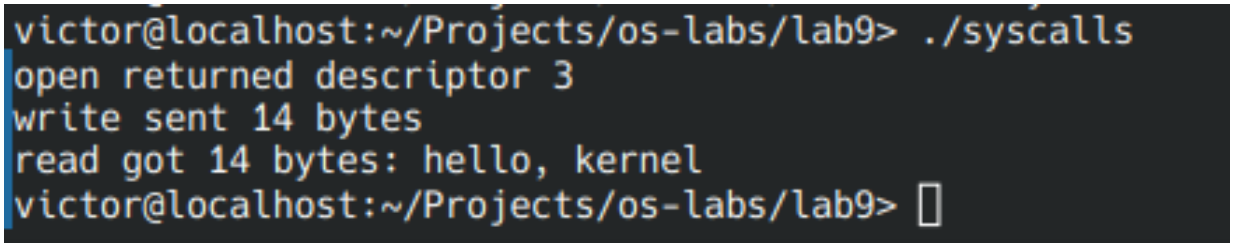



**Figure 2.1 – Output from the direct system-call program**

## 2.2 Interpreting the trace

The trace shows the file operations as `openat`, `write`, `close`, `openat`, `read` and `close`. The C library's `open` wrapper uses Linux's `openat` system call. Before the program accesses `note.txt`, the dynamic loader also opens and reads shared-library files.

The first file descriptor is 3 because descriptors 0, 1 and 2 are already assigned to standard input, standard output and standard error. Linux allocates the lowest available descriptor, so the next open receives 3. Closing it makes the descriptor available again; the read-only open can reuse descriptor 3.

## CONCLUSIONS

`strace` makes the boundary between library-level operations and kernel calls visible. In this run, `write` to descriptor 1 emitted the directory listing, and the filtered command produced 187 trace lines. The file experiment showed how open, write, read and close operate through descriptors, with 3 selected as the lowest free descriptor after the standard streams.